be obtained. Security assurance and security engineering are discussed in more detail in Chapter 18.

Mobile security is a complex subject that must be fully understood before effective security testing can be accomplished.[4] MobileApps and the client-side and server-side environments in which they are housed represent an attractive target for external hackers, disgruntled employees, dishonest competitors, and anyone else who wishes to steal sensitive information, maliciously modify content, degrade performance, disable functionality, or embarrass a person, organization, or business.

Security tests are designed to probe vulnerabilities of the client-side environment, the network communications that occur as data are passed from client to server and back again, and the server-side environment. Each of these domains can be attacked, and it is the job of the security tester to uncover weaknesses that can be exploited by those with the intent to do so.

On the client side, vulnerabilities can often be traced to preexisting bugs in browsers, e-mail programs, or communication software. On the server side, vulnerabilities include denial-of-service attacks and malicious scripts that can be passed along to the client side or used to disable server operations. In addition, server-side databases can be accessed without authorization (data theft).

To protect against these (and many other) vulnerabilities, firewalls, authentication, encryption, and authorization techniques can be used. Security tests should be designed to probe each of these security technologies in an effort to uncover security holes.

The actual design of security tests requires in-depth knowledge of the inner workings of each security element and a comprehensive understanding of a full range of networking technologies. If the MobileApp or WebApp is business critical, maintains sensitive data, or is a likely target of hackers, it's a good idea to outsource security testing to a vendor who specializes in it.

21.8 PERFORMANCE TESTING

For real-time and embedded systems, software that provides required functionality but does not conform to performance requirements is unacceptable. Performance testing is designed to test the run-time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as tests are conducted. However, it is not until all system elements are fully integrated that the true performance of a system can be ascertained.

Nothing is more frustrating than a MobileApp that takes minutes to load content when competitive apps download similar content in seconds. Nothing is more aggravating than trying to log on to a WebApp and receiving a "server-busy" message, with the suggestion that you try again later. Nothing is more disconcerting than a MobileApp or WebApp that responds instantly in some situations and then seems to go into an infinite wait state in other situations. All of these occurrences happen on the Web every day, and all of them are performance related.

4 Books by Bell et al. [Bel17], Sullivan and Liu [Sul11], and Cross [Cro07] provide useful information about the subject.

Performance testing is used to uncover performance problems that can result from a lack of server-side resources, inappropriate network bandwidth, inadequate database capabilities, faulty or weak operating system capabilities, poorly designed WebApp functionality, and other hardware or software issues that can lead to degraded client-server performance. The intent is twofold: (1) to understand how the system responds as *loading* (i.e., number of users, number of transactions, or overall data volume), and (2) to collect metrics that will lead to design modifications to improve performance.

Performance tests are often coupled with stress testing and usually require both hardware and software instrumentation. That is, it is often necessary to measure resource utilization (e.g., processor cycles) in an exacting fashion. External instrumentation can monitor execution intervals, log events (e.g., interrupts) as they occur, and sample machine states on a regular basis. By instrumenting a system, the tester can uncover situations that lead to degradation and possible system failure.

Some aspects of MobileApp performance, at least as the end user perceives it, are difficult to test. Network loading, the vagaries of network interfacing hardware, and similar issues are not easily tested at the client or browser level. Mobile performance tests are designed to simulate real-world loading situations. As the number of simultaneous app users grows, or the number of online transactions increases, or the amount of data (downloaded or uploaded) increases, performance testing will help answer the following questions:

- Does the server response time degrade to a point where it is noticeable and unacceptable?
- At what point (in terms of users, transactions, or data loading) does performance become unacceptable?
- What system components are responsible for performance degradation?
- What is the average response time for users under a variety of loading conditions?
- Does performance degradation have an impact on system security?
- Is app reliability or accuracy affected as the load on the system grows?
- What happens when loads that are greater than maximum server capacity are applied?
- Does performance degradation have an impact on company revenues?

To develop answers to these questions, two different performance tests are conducted: (1) *load testing* examines real-world loading at a variety of load levels and in a variety of combinations, and (2) *stress testing* forces loading to be increased to the breaking point to determine how much capacity the app environment can handle.

The intent of load testing is to determine how the WebApp and its server-side environment will respond to various loading conditions. As testing proceeds, permutations to the following variables define a set of test conditions:

N, number of concurrent users

T, number of online transactions per unit of time

D, data load processed by the server per transaction

In every case, these variables are defined within normal operating bounds of the system. As each test condition is run, one or more of the following measures are collected: average user response, average time to download a standardized unit of data, or average time to process a transaction. You should examine these measures to determine whether a precipitous decrease in performance can be traced to a specific combination of *N, T,* and *D*.

Load testing can also be used to assess recommended connection speeds for users of the WebApp. Overall throughput, *P,* is computed in the following manner:

$$P = N \times T \times D$$

As an example, consider a popular sports news site. At a given moment, 20,000 concurrent users submit a request (a transaction, *T*) once every 2 minutes on average. Each transaction requires the WebApp to download a new article that averages 3K bytes in length. Therefore, throughput can be calculated as:

$$P = \frac{20{,}000 \times 0.5 \times 3\text{kb}}{60} = 500 \text{ Kbytes/sec}$$

$$= 4 \text{ megabits per second}$$

The network connection for the server would therefore have to support this data rate and should be tested to ensure that it does.

Stress testing for mobile apps attempts to find errors that will occur under extreme operating conditions. In addition, it provides a mechanism for determining whether the MobileApp will degrade gracefully without compromising security. Among the many actions that might create extreme conditions are: (1) running several mobile apps on the same device, (2) infecting system software with viruses or malware, (3) attempting to take over a device and use it to spread spam, (4) forcing the mobile app to process inordinately large numbers of transactions, and (5) storing inordinately large quantities of data on the device. As these conditions are encountered, the MobileApp is checked to ensure that resource-intensive services (e.g., streaming media) are handled properly.

21.9 REAL-TIME TESTING

The time-dependent, asynchronous nature of many mobile and real-time applications adds a new and potentially difficult element to the testing mix—time. Not only does the test-case designer have to consider conventional test cases but also event handling (i.e., interrupt processing), the timing of the data, and the parallelism of the tasks (processes) that handle the data. In many situations, test data provided when a real-time system is in one state will result in proper processing, while the same data provided when the system is in a different state may lead to error. In addition, the intimate relationship that exists between real-time software and its hardware environment can also cause testing problems. Software tests must consider the impact of hardware faults on software processing. Such faults can be extremely difficult to simulate realistically.

Many MobileApp developers advocate *testing in the wild*, or testing in the users' native environments with the production release versions of the MobileApp resources

TABLE 21.1

Weighted device platform matrix

		OS1	OS2	OS3
	Ranking	3	4	7
Device1	7	N/A	28	49
Device2	3	9	N/A	N/A
Device3	4	12	N/A	N/A
Device4	9	N/A	36	63

[Soa11]. Testing in the wild is designed to be agile and respond to changes as the MobileApp evolves [Ute12].

Some of the characteristics of testing in the wild include adverse and unpredictable environments, outdated browsers and plug-ins, unique hardware, and imperfect connectivity (both Wi-Fi and mobile carrier). To mirror real-world conditions, the demographic characteristics of testers should match those of targeted users, as well as those of their devices. In addition, you should include use cases involving small numbers of users, less-popular browsers, as well as a diverse set of mobile devices. Testing in the wild is always somewhat unpredictable, and test plans must be adapted as testing progresses. For further information, Rooksby and his colleagues have identified themes that are present in successful strategies for testing in the wild [Roo09].

Because MobileApps are often developed for multiple devices and designed to be used in many different contexts and locations, a *weighted device platform matrix* (WDPM) helps ensure that test coverage includes each combination of mobile device and context variables. The WDPM can also be used to help prioritize the device/context combinations so that the most important are tested first.

The steps to build the WDPM (Table 21.1) for several devices and operating systems are: (1) list the important operating system variants as the matrix column labels, (2) list the targeted devices as the matrix row labels, (3) assign a ranking (e.g., 0 to 10) to indicate the relative importance of each operating system and each device, and (4) compute the product of each pair of rankings and enter each product as the cell entry in the matrix (use NA for combinations that are not available).

Testing effort should be adjusted so that the device/platform combinations with the highest ratings receive the most attention for each context variable under consideration.[5] In Table 21.1, **Device4** and **OS3** have the highest rating and would receive high-priority attention during testing.

Actual mobile devices have inherent limitations precipitated by the combination of hardware and firmware delivered in the device. If the range of potential device platforms is large, it is expensive and time consuming to perform MobileApp testing.

5 *Context variables* are variables that are associated with either the current connection or the current transaction that the MobileApp will use to direct its visible-user behavior.

Mobile devices are not designed with testing in mind. Limited processing power and storage capacity may not allow loading of the diagnostic software needed to record the test-case performance. Emulated devices are often easier to manage and allow easier acquisition of test data. Each mobile network (there are hundreds, worldwide) uses its own unique infrastructure to support the mobile Web. Emulators often cannot emulate the effects and timing of network services, and you may not see problems that users will have when the MobileApp runs on an actual device.

Creating test environments in-house is an expensive and error-prone process. Cloud-based testing can offer a standardized infrastructure and preconfigured software images, freeing the MobileApp team from the need to worry about finding servers or purchasing their own licenses for software and testing tools [Goa14]. Cloud service providers give testers access to scalable, ready-to-use virtual laboratories with a library of operating systems, test and execution management tools, and storage necessary for creating a test environment that closely mirrors the real world [Tao17].

Cloud-based testing is not without potential problems: lack of standards, potential security issues, data location and integrity issues, incomplete infrastructure support, improper usages of services, and performance issues are only some of the common challenges that face development teams that use the cloud approach.

Last, it is important to monitor power consumption specifically associated with the use of the MobileApp on a mobile device. Transmitting information from mobile devices consumes more power than monitoring a network for a signal. Processing streaming media consumes more power than loading a Web page or sending a text message. Assessing power consumption accurately must be done in real time on the actual device and in the wild.

21.10 Testing AI Systems

As we discussed in Chapter 13, mobile users expect products like MobileApps, VR systems, and video games to be context aware. Whether the software product is reacting to the user's environment [Abd16], automatically adapting the user interface based on past user behaviors [Par15], or providing a realistic nonplaying character (NPC) in a game situation [Ste16], artificial intelligence (AI) techniques are involved. Often these techniques make use of things like machine learning, data mining, statistics, heuristic programming, or rule-based systems that are outside the scope of this book. There are several problems common to testing these systems that can be addressed with the techniques we have discussed.

AI techniques make use of information that has been obtained from human experts or summarized from large numbers of observations saved in a data store of some kind. These data need to be organized in some way so that they can be accessed and updated efficiently if the software product is to be context aware or self-adaptive. The heuristics for making use of these data to assist decision making in the software are usually described by humans in use cases or formulas obtained from statistical data analysis. Part of what makes these systems hard to test is the large number of data interactions that need to be accounted for by the software, but whose occurrence is hard to predict. Software engineers often need to rely on simulation and model-based techniques to test AI systems.

21.10.1 Static and Dynamic Testing

Static testing is a software verification technique that focuses on review rather than executable testing. It is important to ensure that human experts (stakeholders who understand the application domain) agree with the ways in which the developers have represented the information and its use in the AI system. Like all software verification techniques, it is important to ensure that the program code represents the AI specifications, which means mapping between use case inputs and outputs is reflected in the code.

Dynamic testing for AI systems is a validation technique that exercises the source code with test cases. The intent is to show that the AI system conforms to the behaviors specified by the human experts. In the case of knowledge discovery or data mining, the program may have been designed to discover new relationships unknown to human experts. Human experts must validate these new relationships before they are used in safety-critical software products [Abd16] [Par15].

Many of the real-time testing issues discussed in Section 21.9 apply in dynamic testing of AI systems. Even if automatically generated simulated test cases are used, it is not possible to test every combination of events the software will encounter in the wild. It is often desirable to build in mechanisms to allow the users to specify when they are not happy with the decisions made by the program and collect information on the program state for future corrective action by developers.

21.10.2 Model-Based Testing

Model-based testing (MBT) is a black-box testing technique that uses information contained in the requirements model (in particular the user stories) as the basis for the generation of test cases [DAC03]. In many cases, the model-based testing technique uses formalism like UML state diagrams, an element of the behavioral model (Chapter 8), as the basis for the design of test cases.[6] The MBT technique requires five steps:

1. **Analyze an existing behavioral model for the software, or create one.** Recall that a *behavioral model* indicates how software will respond to external events or stimuli. To create the model, you should perform the steps discussed in Chapter 8: (1) evaluate all use cases to fully understand the sequence of interaction within the system, (2) identify events that drive the interaction sequence and understand how these events relate to specific objects, (3) create a sequence for each use case, (4) build a UML state diagram for the system (e.g., see Figure 8.8), and (5) review the behavioral model to verify accuracy and consistency.
2. **Traverse the behavioral model and specify the inputs that will force the software to make the transition from state to state.** The inputs will trigger events that will cause the transition to occur.
3. **Review the behavioral model, and note the expected outputs as the software makes the transition from state to state.** Recall that each state transition is triggered by an event and that as a consequence of the transition

6 Model-based testing can also be used when software requirements are represented with decision tables, grammars, or Markov chains [DAC03].